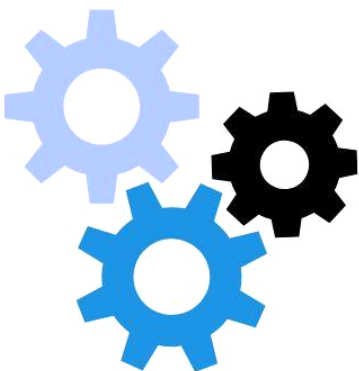


vUAJ

Memoria explicativa



David Rivera
Tyler Zarzuela
Andrea Vega
Muxu Rubia
Eva Feliu



Índice

1. Resumen del proyecto.....	4
2. Objetivos.....	4
3. Diseño e implementación.....	5
3.1. HUD Personalizable.....	5
3.1.1 Pipeline de uso en el jugador.....	5
3.1.2 Implementación.....	6
Instrucción de uso para el desarrollador.....	6
3.2. Audiodescripción (TTS).....	6
3.2.1 Implementación.....	7
Instrucciones de uso de TTSSubtitles.....	7
Instrucciones de uso de TTSHUD.....	7
3.3. Lupa de Pantalla.....	7
3.3.1 Implementación.....	8
Prefab Camera & MagnifyingGlass.....	8
PlaceUIHere: Canvas.....	8
MagnifyingGlass: Camera.....	8
Instrucción de uso para el desarrollador.....	9
3.3.2 Ejemplos.....	9
3.4. Notificaciones Personalizables.....	10
3.4.1. Configuración del usuario.....	10
3.4.2. Gestión centralizada de notificaciones.....	10
3.4.3. Visualización de notificaciones.....	11
3.4.4. NotificationAssetLibrary – Biblioteca de recursos centralizada.....	11
3.4.5. Integración con GameManager.....	11
3.5. Texto accesible (tamaño de letra y tipografías alternativas).....	11
3.5.1 Implementación.....	12
3.6. Percepción de volumen de los objetos.....	12
3.6.1 Implementación.....	12
4. Resultados obtenidos.....	13
5. Conclusiones.....	13
6. Referencias.....	13
7. Adenda.....	15

1. Resumen del proyecto

Vision Unity Accessibility Junction (vUAJ) es una herramienta diseñada para facilitar la integración de opciones de accesibilidad visual en juegos desarrollados con Unity 2D, basada en las pautas establecidas en el [Libro blanco de accesibilidad en videojuegos](#). Su objetivo es mejorar la experiencia de jugadores con necesidades visuales. Entre sus características destacan:

HUD personalizable	Los jugadores pueden ajustar la posición, tamaño y sombras de los elementos del HUD para adaptarlos a sus necesidades visuales.
Audiodescripción	La herramienta <i>Text To Speech</i> (TTS) lee los elementos del juego y los subtítulos en voz alta, ayudando a la comprensión.
Lupa de pantalla	Permite hacer zoom por la pantalla para mejorar la visibilidad de los elementos del juego.
Notificaciones personalizables	El sistema de notificaciones permite a los usuarios ajustar su posición, tamaño, duración y estilo visual. Los desarrolladores también pueden personalizar los colores, iconos, texto, sonidos y feedback háptico de las notificaciones que quieran lanzar.
Tipografías alternativas	Permite cambiar la tipografía del juego a fuentes que faciliten la lectura, especialmente para personas con dislexia.
Aumento del tamaño de letra	Incluye una opción para aumentar el tamaño de los textos en el juego, mejorando su visibilidad.
Percepción de volumen	Resalta el contorno de colisión de los objetos interactivos para facilitar así la navegación del entorno y la localización de dichos objetos.

2. Objetivos

1. Desarrollar un plugin multi-herramienta para Unity que facilite la integración de elementos de accesibilidad visual en videojuegos 2D, orientado especialmente a proyectos de corta duración o con estructuras sencillas, donde la incorporación de soluciones accesibles suele estar limitada por recursos o tiempo.

2. Fomentar la integración temprana de la accesibilidad en el proceso de desarrollo, proporcionando a los desarrolladores un conjunto de herramientas que permitan integrar accesibilidad visual desde las primeras etapas del diseño, sin requerir arquitecturas complejas o especializadas.

3. Garantizar la modularidad del sistema, permitiendo que los distintos módulos funcionen de manera independiente o en combinación, bajo un conjunto de reglas que aseguren su compatibilidad e integración coherente. Esto facilita que los desarrolladores adopten solo los componentes relevantes para sus juegos, optimizando la implementación.

4. Garantizar una experiencia de usuario personalizada y cómoda para personas con dificultades visuales o cognitivas, mediante herramientas que les permitan adaptar libremente la interfaz del juego a sus necesidades reales y mejorar así su autonomía y disfrute. Inspirándonos en las directrices recogidas en el [Libro blanco de accesibilidad en videojuegos](#), asegurando que las herramientas propuestas respondan a necesidades reales y estén alineadas con los estándares actuales de accesibilidad en videojuegos.

3. Diseño e implementación

Entorno de desarrollo:

- **Motor:** Unity 2022.3 LTS
- **Lenguaje de programación:** C#
- **Repositorio del proyecto:** [vUAJ](#)

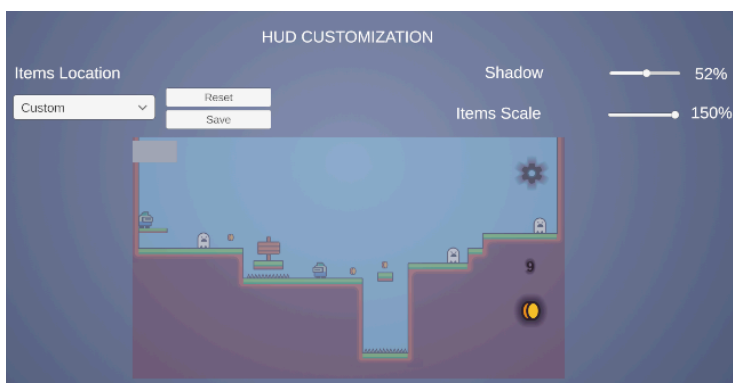
Todas las herramientas desarrolladas cuentan con un **menú de personalización genérico**, al que se puede acceder desde la escena *Settings* en la plantilla de *Unity* ofrecida. Este menú es capaz de guardar la información de configuración que el usuario haya decidido gracias a un sencillísimo **sistema de serialización**.

Este consiste de una única clase, *Serializer*, que se encarga de coger un objeto, serializarlo a un **archivo JSON**, y deserializarlo utilizando *JsonUtility*, que es una clase estática dada por *Unity*. De esta forma, cada objeto serializable guarda su propio archivo JSON con el nombre que se le quiera asignar. Toda esta información se guarda en *Application.persistentDataPath*, para que se mantenga entre partidas. La información a serializar es un *struct* con valores públicos que el desarrollador decide. Este formato facilita la implementación de la serialización.

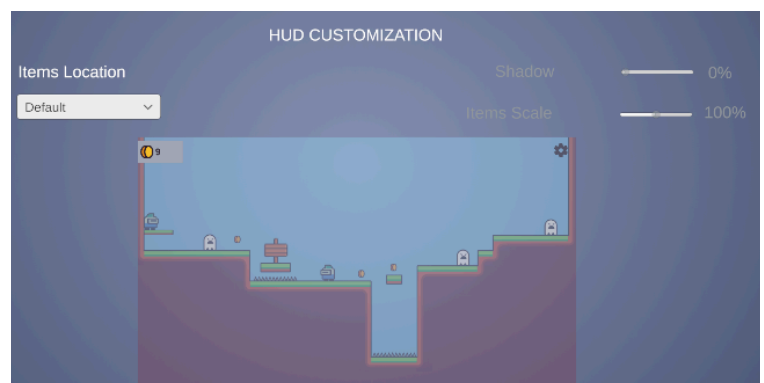
3.1. HUD Personalizable

El HUD es la información que se muestra en pantalla durante la partida, como puede ser la puntuación o vidas del jugador. Con esta herramienta habilitamos que el jugador pueda modificar tres aspectos de los elementos del HUD-in-game: el **tamaño**, el **sombreado** y la **posición**. Hemos seleccionado estas características contemplando problemas relacionados con la pérdida de campo visual, como puede ser la pérdida de visión periférica o de uno de los ojos. Con el tamaño y el sombreado parametrizable, los elementos del HUD persistentes pueden resaltar sobre el resto del juego, y así potencialmente reducir el agotamiento visual del jugador o el uso repetido de la lupa.

Proporcionamos al jugador dos tipos de HUD: el **Default**, el proporcionado por el desarrollador del juego, y el **Custom**, la versión que podrá modificar el jugador.



Editor de Custom HUD



Visualización de Default HUD

3.1.1 Pipeline de uso en el jugador

El jugador puede acceder al menú de setting y modificar, seleccionando la opción *custom*, su menú. El HUD seleccionado y tras dar al botón de guardar, será el que se encuentre en el juego.

Cabe especificar que los elementos del HUD pueden adoptar **cualquier posición dentro de los límites de la pantalla**. No son tenidas en cuenta las colisiones entre elementos del HUD, pero permitimos que el jugador resetee

su customización a su última versión guardada (y si esta no existe, al *default*) por si se equivoca o desea volver a empezar mediante el botón de *reset*. Podrá guardar todos sus cambios clicando sobre el *save*.

Una vez comenzada la sesión, el jugador podrá cambiar en cualquier momento su selección de HUD (custom o default) accediendo al menú de *Settings* de accesibilidad.

3.1.2 Implementación

La clase principal del HUD customizable es *HUDManager*. Este tiene una referencia al prefab del HUD proporcionado por el desarrollador del juego y se encarga de serializar y almacenar tanto el HUD customizado como el por defecto para poder recuperarlo durante el juego.

Instrucción de uso para el desarrollador

- El desarrollador del juego deberá asignar manualmente el *prefab* del HUD deseado al *prefab* del objeto *HUDManager* para poder llevar a cabo las visualizaciones en *Settings*.
- Después deberá llamar al comienzo del juego a la función *loadCurrentHUD* de la instancia del *Manager*. Hemos habilitado en el juego de ejemplo un script *GameController* en la escena de juego que tiene referencias de recursos necesarios que deben pasarse a los managers al comenzar el juego.
- (+) Sí lo desea, puede modificar la imagen del panel de visualización de pantalla con un fondo para dar perspectiva. No es recomendable incluir el fondo en el prefab del HUD pues podría sufrir desplazamiento.

El método de guardado reside en *saveConfig()*: a partir de un diccionario de datos aportado o presente en el manager (el correspondiente a custom) guarda la información de un tipo de HUD en formato **JSON**.

```
{
  "CoinText":{"baseScale":{"x":1.0,"y":1.0,"z":1.0},"scaleFactor":1.0,"shadowFactor":0.0,"pos":{"x":-822.2000122070313,"y":424.0,"z":0.0}},
  "IconImage":{"baseScale":{"x":1.0,"y":1.0,"z":1.0},"scaleFactor":1.0,"shadowFactor":0.0,"pos":{"x":-871.0,"y":423.9999694824219,"z":0.0}},
  "SettingsButton":{"baseScale":{"x":1.0,"y":1.0,"z":1.0},"scaleFactor":1.0,"shadowFactor":0.0,"pos":{"x":893.0,"y":481.0,"z":0.0}}
}
```

Ejemplo de guardado de datos

Los HUDs podrán cargarse o modificarse con *applySavedConfigToHUD()*.

La segunda clase es *HUDCustom*. Está es la que obtiene la información necesaria para la visualización del hud *custom* y *default* en el menú de *settings*, contiene todos los botones, *sliders* y *prefabs* creados en runtime y se encarga de actualizar los cambios en el *HUDManager* y llamando su *Serialize*. El script de las sombras empleado, *DropShadow*, no es el shadow básico de Unity, sino que ha sido obtenido de forma externa e incluye la posibilidad de añadir offset e iteraciones (aunque finalmente no se haya incluido como opción de parametrización en el menú).

Finalmente encontramos la clase *Drag*, que poseen todos los elementos del visualizable custom en el menú de settings para poder moverse entre los límites de la pantalla (mediante las funciones *OnDrag()*, que obtiene el punto en pantalla y desplaza el objeto y *ClampToParent()*, que limita el movimiento dentro del margen del panel que representa la pantalla.

3.2. Audiodescripción (TTS)

El sistema de **audiodescripción** o TTS (*text-to-speech*) consta de varias clases. La principal es *TTSTManager*, que se encarga de incorporar la *DLL de terceros* que hemos usado para la lectura del texto, así como de gestionar la activación/desactivación del sistema y hacer la llamada para **transformar texto en voz**. Con la incorporación de

esta funcionalidad se busca la accesibilidad para personas con ceguera parcial y total. De esta manera, **no hace falta que lean lo que hay en pantalla** sino que la podrán escuchar. A continuación, se revisan las diferentes clases utilizadas.

3.2.1 Implementación

- **TTSTNotifications:** Esta clase se encarga de llamar a TTSTManager y leer el texto de la notificación que se le pase mediante el método *SayNotification*. Tiene la opción implementada de repetición del mensaje por si este no se ha escuchado o no ha quedado claro pulsando una tecla programable. Por defecto es la 'R'.

- **TTSSubtitles:** Esta clase se encarga de leer secuencialmente los mensajes recogidos en un JSON en un formato específico para la lectura e implementación de subtítulos. Guarda las cadenas de texto que componen cada interacción y las reproduce conforme se pulsa un botón o se le indica a la clase. Para la lectura y secuenciación de los diálogos hace uso de la clase *JSONParser*.

Instrucciones de uso de TTSSubtitles

TTSSubtitles leerá un JSON situado por defecto en *Application.persistentDataPath* que debe tener el siguiente formato:

```
{
    "subtitles": [
        "Personaje 1:" esto es una línea de dialogo.",
        "Personaje 2:" esto es otra línea de dialogo.",
        "Personaje 3:" esto es la última línea de dialogo."
    ]
}
```

Es importante destacar que el indicador de objeto de JSON (en este caso "Subtitles") debe ser el mismo que el único atributo que pertenece a la clase *StringList* que se encuentra dentro de *JSONParser*. Por lo que en todos los casos, debe llamarse de esta manera en los diferentes JSON que el desarrollador haya creado con los subtítulos.

A continuación tan solo hace falta llamar al método *ReadNextSubtitle()* de *TTSSubtitles* cada vez que se quiera leer una nueva línea de diálogo, haciendo posible la incorporación de un botón o tecla que avance así con los subtítulos.

A su vez, esta clase incorpora un método para cambiar el JSON que contiene la lectura de los diálogos durante la ejecución llamándose *ChangeSubtitles(string newFileName)*. De esta forma, pueden guardarse diferentes archivos JSON con distintos diálogos.

- **TTSHUD:** Esta clase se encarga de leer el texto asociado tanto a elementos tanto del HUD como del juego (aunque en ese caso la cámara de la escena tendrá que tener un *PhysicsRaycaster2D*)

Instrucciones de uso de TTSHUD

Para poder usarse se debe añadir como componente a aquellos elementos de la UI que contengan texto o que se quieran leer en voz alta. Dichos elementos deben tener un componente *TextMeshPro* o *TextMeshProUGUI* del que se obtendrá el texto.

3.3. Lupa de Pantalla

Para personas con visibilidad reducida, es posible que mucho de lo que un videojuego muestre en pantalla se le haga pequeño. **Con la ayuda de una lupa**, es posible que se **mejore la experiencia de dichos usuarios**. Esta

consiste en **una sección ampliada de la pantalla**, que puede moverse, ampliarse e incluso reducirse. También forma parte del *Libro blanco de accesibilidad para desarrolladores*, marcándose como una implementación de 2 estrellas de dificultad. La solución propuesta por el libro es la de **utilizar una cámara adicional**, pero más cercana a la pantalla. Hemos optado por implementarlo de esta manera, ya que es más sencillo, compatible y asequible que la utilización de *shaders*.

3.3.1 Implementación

Surgieron una serie de **problemas** durante la implementación, ya que al utilizar una cámara como forma de ampliar la pantalla, **esta no es capaz de renderizar la interfaz de usuario** que se encuentre en un *Canvas* en *Screen Space*, ya que no son renderizados en ninguna cámara, sino por encima de esta. Por lo tanto, al tratar de hacer *zoom* sobre los elementos de UI, estos no aparecían.

La solución a la que se llegó fue **que todos los *Canvas* que quieran ser ampliables mediante una lupa, deben estar en modo *WorldSpace***, para que formen parte del mundo y puedan ser renderizados por la lupa. Como consecuencia, todas aquellas cámaras que quieran mostrar la UI, deben situarse por delante del mismo. Además, se creó un script llamado *AutoWorldCanvasSetup* que automatiza el cambio en los *canvas* de *Overlay* a *WorldSpace*, permitiendo al resto de miembros del equipo que habían comenzado a trabajar en una configuración distinta seguir con su trabajo sin preocuparse de cambiar todo.

El usuario puede cambiar las teclas para adaptarlas a sus necesidades en el menú de *Settings* mediante el *script* *InputRebinderUI*, el cual permite gestionar el mapeo de teclas.

Prefab Camera & MagnifyingGlass

Como resultado de la implementación, tenemos un *prefab* (*Camera&MagnifyingGlass*) que contiene la que sería la cámara principal del juego, además de dos hijos, que se detallarán a continuación.

PlaceUIHere: Canvas

Es un *canvas* genérico en *WorldSpace*, que ya está a la distancia perfecta de la cámara. No es más que un *placeholder*, ya que se puede borrar y todo funcionará correctamente; es tan solo una indicación de dónde debería la UI que queramos que sea ampliable con la lupa.

MagnifyingGlass: Camera

Es la lupa como tal. Tiene una cámara, además de nuevos componentes necesarios para su funcionamiento:

- ***MagnifyingGlassController***: Encargado de controlar la lupa. Su función principal es activar, desactivar, mover, ampliar y reducir el *zoom* en función de sus *InputActions*, que son un objeto serializado que el desarrollador puede cambiar como considere. Por defecto, se implementa para que pueda (des)activarse con **z**; reducirse con **q**; ampliarse con **e**; y moverse con **w**, **a**, **s**, **d**. Cuando desactivamos la lupa, deja de recibir los demás eventos, para evitar confusión. Este *script* no bloquea el *input* recibido por el juego, por lo que si se utilizan las mismas teclas mencionadas para alguna función del juego es imprescindible que se cambien. Para evitarlo, el componente cuenta con dos *eventos* que son llamados cuando se activa o desactiva la lupa, respectivamente. Ahí el desarrollador puede suscribir funciones a esos eventos que bloqueen *input* para otras cosas que no sean la lupa mientras esta se utilice.
- ***FixMovementToCamera***: Consigue los bordes de una cámara en posición de mundo y fija el *transform* de un objeto a que no se pueda salir de estos. Este objeto plantea ser genérico, pero no ha sido sometido a pruebas suficientes para probarlo. Se utiliza para que la lupa no se salga de la cámara principal, para evitar que el usuario la pierda de vista.

Además de estos componentes, también cuenta con dos hijos, que completan la funcionalidad mencionada:

- **Background:** Un *canvas* que actuará como fondo para la ampliación de la pantalla, para que se pueda diferenciar correctamente de la zona no ampliada. No deja de ser un *sprite*, por lo que su personalización es posible.
- **RenderTexture:** El siguiente hijo es otro *canvas*, pero que contiene una imagen sobre la que se plasma una *RenderTexture*, que es la que nos devuelve el resultado de lo que la cámara de la *lupa* graba en la pantalla. La plasmamos en una imagen para que el jugador pueda verla.

Instrucción de uso para el desarrollador

La inclusión de la lupa en un proyecto 2D es bastante sencilla, ya que cuenta con el *prefab* anteriormente descrito que facilita mucho la configuración. El único requerimiento es la sustitución de la cámara que se esté utilizando en el proyecto por la del *prefab*. Incluso, podría no sustituirse y tan solo incluir el hijo *MagnifyingGlass:Camera*, que es el que realmente contiene la funcionalidad. Sí que es necesario, sin embargo, que toda la UI que el desarrollador quiera que sea ampliable por la *lupa* se adapte a las necesidades anteriormente especificadas: que sea hijo de la cámara principal (al mismo nivel que el *MagnifyingGlass:Camera*) y que esté en *WorldSpace*, por delante de la cámara. Una vez cumplidos dichos requisitos, la lupa ya es perfectamente utilizable.

3.3.2 Ejemplos

A continuación, se muestran dos ejemplos de utilización de la lupa:

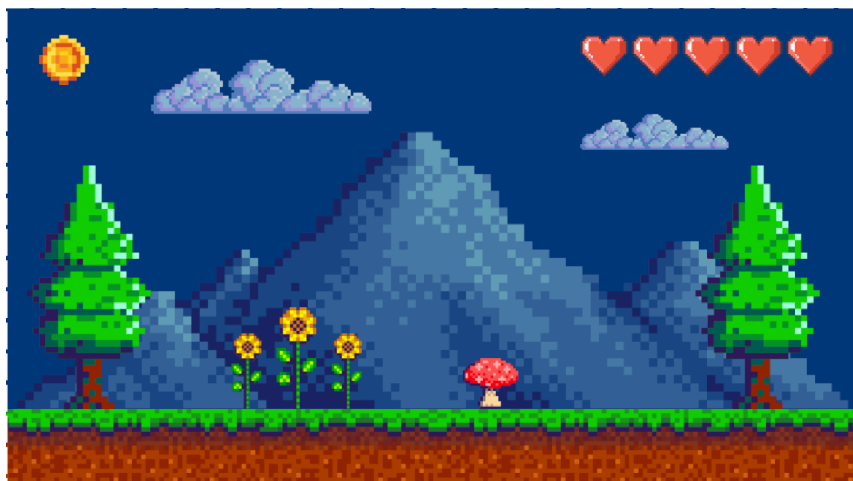
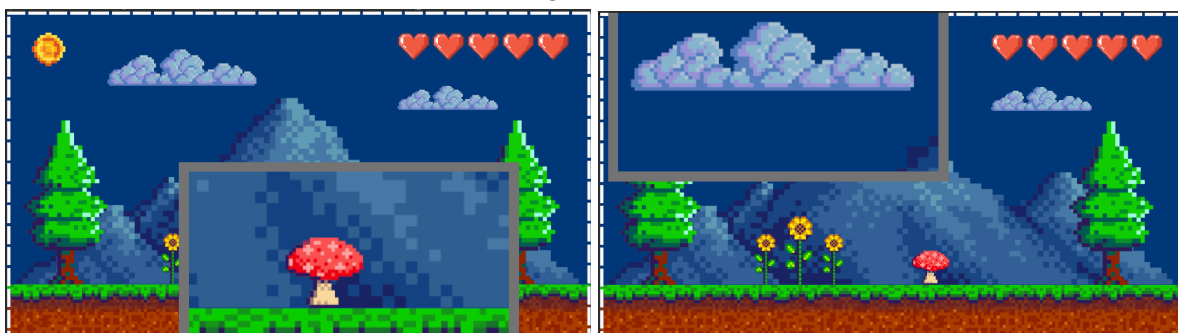


Imagen de base



Lupa siendo utilizada sobre la imagen de base. Se muestra rodeada de un marco grisáceo

3.4. Notificaciones Personalizables

La personalización de las notificaciones resulta esencial para personas con visión parcial o sensibilidad visual, ya que mejora notablemente la comprensión de la información del juego sin depender exclusivamente del canal visual. Este sistema, como el resto, se encuentra alineado con las recomendaciones del [Libro Blanco de Accesibilidad para Desarrolladores](#), y permite que los jugadores ajusten la forma en que reciben los mensajes según sus necesidades visuales o preferencias cognitivas.

El sistema de notificaciones implementado constituye una herramienta versátil tanto para los **usuarios** como para los **desarrolladores**. Desde el punto de vista del jugador, permite recibir información de forma accesible y adaptada a sus preferencias. Para el desarrollador, facilita el envío de notificaciones desde cualquier parte del juego mediante un único método centralizado, sin preocuparse por los detalles de presentación.

3.4.1. Configuración del usuario

Desde la escena de configuración, el jugador puede adaptar la experiencia visual y sonora a sus necesidades y preferencias mediante una interfaz accesible e intuitiva. Los parámetros configurables son los siguientes:

- **Tipo de notificación:** puede ser estándar, visible para todos los jugadores como parte del flujo habitual del juego, o accesible, una notificación adicional que el programador puede activar en puntos concretos del juego para asistir a personas con dificultades visuales.
- **Posición en pantalla:** permite ubicar las notificaciones en diferentes zonas predefinidas.
- **Tamaño:** el usuario puede elegir entre tres tamaños distintos (pequeño, mediano o grande).
- **Duración:** las notificaciones pueden configurarse para desaparecer automáticamente tras un tiempo definido (limitadas) o permanecer en pantalla hasta que el jugador realice una acción (indefinidas).
- **Estilo visual:** se ofrece la posibilidad de seleccionar entre distintos estilos de fondo: negro opaco, translúcido o con efectos de brillo.

Estos ajustes se gestionan mediante una combinación de controles de UI como carruseles personalizados, botones y menús desplegables. Todos los cambios realizados en esta escena quedan registrados en el *NotificationManager*, asegurando que el comportamiento y la apariencia de las notificaciones se adapten automáticamente a las preferencias del jugador en tiempo real.

3.4.2. Gestión centralizada de notificaciones

Internamente, el *NotificationManager* centraliza el envío de notificaciones a través de un único método público que acepta como entrada una estructura *NotificationData*. Esta estructura encapsula todos los elementos necesarios para construir una notificación completa de forma clara y estructurada.

```
public enum NotificationPosition { TopCenter, TopLeft, TopRight, BottomLeft, BottomRight }
public enum NotificationSize { Small, Medium, Large }
public enum NotificationDuration { Seconds3, Seconds5, Seconds7, Seconds10, Seconds15, Seconds20, Unlimited }
public enum NotificationStyle { Normal, Translucent, Black, Glow }
public enum NotificationLevel { Standard, Assisted }
public enum HapticFeedbackType { None, Light, Medium, Heavy, Pulse }
public struct NotificationData
{
    public string message;
    public string iconName;
    public Color color;
    public string soundName;
    public HapticFeedbackType haptic;
    public NotificationPosition position;
    public NotificationSize size;
    public NotificationStyle style;
    public NotificationDuration duration;
}
```

El método *SpawnNotification* se encarga de procesar esta información, validarla y completarla, en caso necesario, con las preferencias del usuario previamente almacenadas en memoria. Estas preferencias se sincronizan automáticamente desde la escena de configuración y actúan como valores por defecto, evitando que el programador deba repetir configuraciones en cada notificación. Además, el *NotificationManager* interactúa directamente con el

módulo *NotificationAssetLibrary*, accediendo dinámicamente a los recursos audiovisuales (como iconos o sonidos) según el nombre proporcionado. Finalmente, una vez interpretada y completada toda la información, se delega la visualización a una instancia del componente *NotificationUI*, que se encarga de representar la notificación en pantalla aplicando los efectos visuales, sonoros y hápticos indicados.

3.4.3. Visualización de notificaciones

El componente *NotificationUI* representa la notificación en pantalla. Es un prefab reutilizable que se instancia dinámicamente cada vez que el *NotificationManager* solicita mostrar un nuevo mensaje.

Al activarse, esta clase toma los parámetros visuales definidos por el usuario y los aplica al layout de la notificación. De forma interna, se encarga de cargar el texto, asignar el icono correspondiente, aplicar el estilo visual definido, reproducir el sonido si está habilitado y, si es el caso, activar el feedback háptico mediante el controlador de vibración. El componente también gestiona la animación de entrada y salida de la notificación. Estas transiciones se ajustan automáticamente según la duración definida por el usuario. En caso de que la duración sea indefinida, el sistema permite cerrarla mediante la tecla 'Esc', como una notificación persistente. Adicionalmente, *NotificationUI* está diseñado para ser completamente modular: se adapta a cualquier canvas y escala automáticamente a la resolución de pantalla.

3.4.4. NotificationAssetLibrary – Biblioteca de recursos centralizada

Esta clase cumple la función de centralizar todos los recursos gráficos y sonoros necesarios para las notificaciones. Utiliza el sistema de *Resources.LoadAll()* para escanear y cargar automáticamente todos los íconos e indicadores auditivos disponibles y los almacena en diccionarios internos que permiten el acceso por nombre o tipo, lo que facilita solicitar un recurso con una simple referencia textual, sin preocuparse por rutas o asignaciones manuales.

Este enfoque no solo optimiza el rendimiento al evitar cargas repetidas, sino que permite a los diseñadores modificar o ampliar la biblioteca sin necesidad de editar código. Basta con añadir un nuevo icono o sonido a la carpeta correspondiente para que esté disponible automáticamente en el sistema.

3.4.5. Integración con GameManager

Para cubrir casos en los que ciertas notificaciones requieren pausar el juego (por ejemplo, advertencias críticas o instrucciones importantes), se ha desarrollado un *GameManager* base como singleton. Este componente puede ser extendido por el programador y sirve como punto central de control del estado general del juego. Está conectado directamente al *NotificationManager*, de modo que ciertas notificaciones pueden activar la pausa automáticamente. Para garantizar un funcionamiento coherente, los objetos de la escena deben consultar el estado de pausa antes de ejecutar su lógica, tal como se documenta en el código base.

3.5. Texto accesible (tamaño de letra y tipografías alternativas)

Hemos implementado dos herramientas principales relacionadas con la **accesibilidad visual relativa a textos**:

- **Customización del tamaño de letra:** que permite al usuario aplicar varios *tiers* de aumento en el tamaño de la letra del juego, según sus necesidades.
- **Elección de tipografías alternativas de fácil lectura:** que permite al usuario activar un modo *Dislexia-friendly* y elegir una fuente entre una selección *especialmente escogida por su legibilidad para personas con dislexia* que se aplique al juego.

3.5.1 Implementación

El sistema de accesibilidad de texto integra principalmente 3 *scripts*:

- ***TextAccessibilityManager***: Es el manager (*singleton*) de las opciones de accesibilidad de textos. Incluye los métodos usados para actualizar las variables que los textos usan para determinar su tamaño de letra, o si se debe aplicar la fuente apta para dislexia seleccionada.
- ***FontDropdown***: Es un componente que recibe las fuentes que van a estar disponibles como tipografías de fácil lectura, y las muestra como opciones en un *Dropdown*. Al ser una elegida, le pasa la información al *manager*.
- ***FontOptions***: Es el componente que se debe poner a todos los textos que se desea que se vayan a ver afectados por éstas opciones de accesibilidad (tamaño de letra y tipografía alternativa).

Se recomienda comprobar todos los textos a los que se añada *FontOptions* en el tamaño máximo disponible, y **adaptar** el layout de la pantalla a este tamaño de letra para que sea **verdaderamente accesible**.

3.6. Percepción de volumen de los objetos

Según el **Libro blanco de accesibilidad para desarrolladores**, esta medida de accesibilidad ayuda a distinguir los objetos, personajes u otros elementos del videojuego por volumen, así que hemos integrado esta herramienta que resalta los colliders de los elementos del juego que la incorporen mediante una malla de color.

3.6.1 Implementación

Esta medida de accesibilidad integra principalmente 2 *scripts*:

- ***VolumePerceptionManager***: Es el manager (*singleton*) de la opción de accesibilidad de percepción de volumen. Incluye el método usado para actualizar las variable que los *gameObjects* usan para determinar si deben activar su malla de color para mostrar su collider, y un color personalizable desde el editor para indicar el color que deberán tener éstas.
- ***VolumePerception***: Es el componente que se debe poner a todos los *gameObjects* que se desea que se vayan a ver afectados por ésta opción de accesibilidad. Se encarga de crear un objeto (*border*) al que añade un mesh con la misma forma y tamaño que el collider del *gameObject*.

Se recomienda al desarrollador que escoja un color de alto contraste con respecto a su juego para que sea verdaderamente útil para los jugadores que necesiten esta medida.

4. Resultados obtenidos

Los resultados obtenidos se alinean con los objetivos planteados al inicio del proyecto. Se ha desarrollado con éxito un plugin multi-herramienta para Unity 2D. El sistema permite a los desarrolladores incorporar funciones clave sin requerir arquitecturas complejas ni conocimientos especializados.

El proyecto final se presenta como dos paquetes de Unity, uno que contiene la implementación completa y otro que añade un juego de ejemplo. La carpeta 'vUAJ' que lo forma, está organizada modularmente, en la que cada herramienta se encuentra en su propio directorio independiente. Esto permite utilizar directamente los componentes deseados sin tener que depender de otros y pudiendo borrar o no importar los demás; favoreciendo una integración

rápida y flexible y cumpliendo el objetivo de modularidad, permitiendo que los desarrolladores adapten fácilmente la herramienta a las necesidades específicas de sus juegos. Además, se ha incluido un archivo README.md detallado que guía al desarrollador en el uso de cada módulo, tanto de forma individual como combinada.

Asimismo, el sistema ofrece un alto grado de personalización para los usuarios finales, especialmente aquellos con dificultades visuales o cognitivas. Permite modificar el HUD, escuchar el contenido en pantalla mediante TTS, ajustar notificaciones y ampliar zonas mediante lupa, entre otras opciones. No obstante, se identifican algunas limitaciones:

- Falta de soporte completo para múltiples resoluciones
- La necesidad de usar *World Space* para que la lupa funcione con elementos UI
- Aunque los volúmenes de los objetos en pantalla pueden ser resaltados con colores personalizados, la elección del color sólo puede ser definida por el desarrollador, no por el usuario final en runtime
- Por falta de tiempo, las teclas relacionadas con los subtítulos y el TTS, el usuario no puede remapearlas como debería

Aun así, la herramienta cumple con los estándares actuales de accesibilidad recogidos en el *Libro Blanco de Accesibilidad para Desarrolladores* y proporciona una base sólida para futuros desarrollos.

5. Conclusiones

En conclusión, trabajar en el desarrollo de esta herramienta nos ha ayudado a entender el gran trabajo que requiere crear productos realmente accesibles, y lo necesario que es para algunas personas que se las tenga en cuenta desde el minuto 0 del desarrollo.

A pesar de ésto, y las limitaciones ya mencionadas, creemos que **vUAJ** es realmente útil para facilitar la integración de medidas de accesibilidad en proyectos de Unity. Nos hemos preocupado de comentarla extensiblemente y añadir un README para desarrolladores, además de facilitar el acceso a las medidas implementadas mediante menús legibles y user-friendly.

Aunque por limitaciones de tiempo y falta de acceso a un grupo de usuarios que cumpliesen las características de nuestro público objetivo no nos ha sido posible hacer una prueba formal, nos quedamos con muchas ganas de haberlo hecho, para confirmar la utilidad y correcta implementación de nuestras herramientas.

6. Referencias

- Keijiro, UnityRustTts, Repositorio online: <https://github.com/keijiro/UnityRustTts>
- Libro Blanco de Accesibilidad para Desarrolladores de Videojuegos: https://www.aevi.org.es/web/wp-content/uploads/2023/06/libro_blanco_de_accesibilidad_para_desarrolladores_digital2023_acf.pdf
- Carruseles de las preferencias de notificaciones: Pablo Gozávez, Unity Assets Store https://assetstore.unity.com/packages/tools/gui/scroll-carousel-306533?srltid=AfmBOornHekEfCzShLaWwNsVRpIdQC8Pmx_dxGJHoqRNjmc2U_oRrula
- Investigación de fuentes *Dislexia-Friendly*: https://dyslexiahelp.umich.edu/sites/default/files/good_fonts_for_dyslexia_study.pdf

7. Adenda

Aquí se resume el trabajo realizado por cada uno de los integrantes del equipo:

David Rivera Martínez: Realización de la herramienta entera de **Lupa de pantalla**, serialización y deserialización de todas las herramientas (excepto HUD personalizable), ayuda puntual al resto de miembros del equipo y participación en la memoria del proyecto.

Tyler Zarzuela Amor: Realización de la escena de **Settings** (unión de todos los canvas necesarios para el funcionamiento y explicación de cada herramienta individual), la herramienta entera de **Notificaciones**, realización del *AutoWorldCanvasSetup*, creación del *InputRebinderUI*, código necesario de *gamePlay* para incluir las notificaciones y participación en la memoria del proyecto.

Eva Feliu Aréjola: Realización de la herramienta entera de **HUD customizable**, serialización y deserialización del mismo y participación en la memoria del proyecto

Muxu Rubia Luque: Realización de la herramienta de **Audiodescripción (TTS)** a excepción de la clase *TTSHUD*. Participación en la memoria del proyecto.

Andrea Vega Saugar: Realización de la escena de **Game** y **Main Menu**. La herramienta de **Percepción de Volumen** y la de **Aumento del tamaño de letra** y **Tipografías alternativas**, además de la clase *TTSHUD*. Adaptación de todos los textos y escenas a los distintos tamaños de letra y TTS. Ayuda puntual en la serialización y participación en la memoria del proyecto